

S3データ不着検知時のクライアントへのメール通知

- **ID:** client-email-notification
- **Version:** 0.4.0
- **Created at:** 2026-05-25
- **Authors:** scrumsign-takuyakimura
- **Dependencies:** cross-account-architecture, alarm-naming-convention
- **Status:** draft

背景・目的

S3 へのデータ不着は検知できているが、通知先は社内 Discord のみでクライアントへの直接通知手段がない。データ送信者（クライアント）がメールで早期に気づけるようにする。

全体概要

既存の Discord 通知を **Channel** インターフェースに抽象化し、メール通知チャネルを並列に追加する。エラー種別（error_id）ごとにどのチャネルへ通知するかを **config/error-routing.yml** で定義し、送信先メールアドレスのグループを **config/email.yml** で管理する。

Discord はこれまでどおり社内向け通知として機能し続ける。クライアントはメールで通知を受け取るが、Discord の存在を知る必要はない。

ルーティングアルゴリズム概要

通知のディスパッチは以下の3段階で解決する:

```
alarm_name + log_rows (Insights クエリ結果)
|
▼ Step 1: _resolve_error_id(alarm_name, log_rows) ← ログ有無でコード直書き判定
error_id (例: "s3_data_missing" または "lambda_failure")
|
▼ Step 2: error-routing.yml を引く
channel_ids (例: ["discord", "email.dev", "email.sakura"])
|
▼ Step 3: channel_ids から Channel インスタンスを生成 → 各
Channel.send(message)
通知送信
```

_resolve_error_id は **log_rows** の有無で **error_id** を決定する:

- **hdw-*** アラーム かつ ログ 0 件 → **"s3_data_missing"** (Lambda 未起動 = S3 データ未着)
- **hdw-*** アラーム かつ ログあり → **"lambda_failure"** (Lambda 起動・処理失敗)

email.<group_id> 形式の **channel_id** は **email.yml** の **id** フィールドで定義されたグループを参照する。各段階でエントリが見つからない場合はフォールバックし、WARNING ログを出して継続する。いずれ

かのチャンネルが例外を出しても他チャンネルへの送信は継続する。

ログ件数の有無にかかわらず Bedrock は常呼び出し、`Message` を構築して `_dispatch` に渡す。

主要 definitions

用語	説明
データ不着	期待される時刻までに S3 への対象データが届かない状態
クライアント	データ送信者（船舶オペレーター等）。メール通知の受信者
通知メール	データ不着を検知した際にクライアントへ送るメール。ship_name と未着確認時刻を含む
Channel	通知送信先を抽象化したインターフェース。 <code>id: str</code> と <code>send(message: Message)</code> を持つ
channel_id	Channel の識別子。 <code>"discord"</code> または <code>"email.<group_id>"</code> 形式
メールグループ	<code>email.yaml</code> で定義された送信先アドレスの集合。 <code>id</code> フィールドで識別する
error_id	エラー種別の識別子（例: <code>s3_data_missing</code> ）。 <code>error-routing.yaml</code> でチャンネルと紐付ける
Message	チャンネル非依存の通知データ構造。各 Channel が内部でフォーマットに変換して送信する
ルーティング	<code>alarm_name</code> → <code>error_id</code> → <code>channel_ids</code> → <code>Channel.send()</code> の3段階解決処理
フォールバック	各解決段階で対象エントリが見つからない場合に WARNING ログを出しつつ安全な代替動作に切り替えること

Requirements

REQ-001: Amazon SES でメールを送信する

メール送信手段として Amazon SES を使用する。Lambda 実行ロールの IAM 権限で認証し、クライアント側の事前操作なしに任意アドレスへ送信できる。

詳細な選定根拠は RESEARCH-001.md 参照。

AC:

- AC-001-1: Lambda 実行ロールに `ses:SendEmail` / `ses:SendRawEmail` 権限が付与されている
- AC-001-2: SES サンドボックスが解除されており、任意の外部アドレスへ送信できる
- AC-001-3: 送信元アドレスが `@scrumsign.com` ドメインから送られる（O-2 確定後）

REQ-002: アラーム発火時にクライアントへメールを送る

hdw-* アラームが発火したとき、`error_id` (`s3_data_missing` または `lambda_failure`) に応じたチャンネルへ通知が送られる。`error-routing.yaml` にメールチャンネルが登録されていれば、ログの有無にかかわらずメールが送信される。一方のチャンネルが失敗しても他チャンネルへの送信は継続される。

AC:

- AC-002-1: `error-routing.yml` に登録されたアドレスリスト全件にメールが送信される
- AC-002-2: `SESEmailChannel.send()` が例外を送出しても `DiscordChannel.send()` は実行される

REQ-003: 通知メールに船舶名と未着確認時刻を含める

通知メールの本文に、対象の `ship_name` とアラーム発火時刻 (JST) が含まれる。

AC:

- AC-003-1: メール本文に `ship_name` が含まれる
- AC-003-2: メール本文にアラーム発火時刻が含まれる (JST 表記)

REQ-004: メール の HTML テンプレートを実装する

クライアントが受け取るメールを HTML 形式で送信する。plain text フォールバックも同時に送信する。

AC:

- AC-004-1: SES へのリクエストに `Body.Html` と `Body.Text` の両方が含まれる
- AC-004-2: HTML メールに `ship_name` ・未着確認時刻・severity・推奨アクションが含まれる

REQ-005: エラー種別ごとに通知チャンネルを設定ファイルで制御できる

`config/error-routing.yml` の編集のみで、エラー種別ごとの通知チャンネルを変更できる。コードの変更・再ビルドは不要とする。

AC:

- AC-005-1: `error-routing.yml` の `channels` リストを変更するとディスパッチ先が変わる
- AC-005-2: 対象の `error_id` エントリが存在しない場合、`discord` のみにフォールバックし WARNING ログが出る

REQ-006: メールアドレスグループを設定ファイルで定義し、エラー種別ごとに送信グループを指定できる

`config/email.yaml` にメールアドレスのグループを定義し、`error-routing.yml` の `channels` から `email.<group_id>` 形式で参照する。`email.yaml` の編集のみで送信先を変更でき、コードの変更・再ビルドは不要とする。

`email.yaml` はリスト of dict 形式 (`[{id: group_id, add: [address, ...]}]`) とする。

AC:

- AC-006-1: `email.yaml` のグループのアドレスリストを変更するとそのグループの送信先が変わる
- AC-006-2: `error-routing.yml` に `email.dev` と指定された場合、`email.yaml` の `id: dev` エントリのアドレスリストが使われる
- AC-006-3: `email.<group_id>` の `group_id` が `email.yaml` に存在しない場合、送信をスキップし WARNING ログが出る

REQ-007: ルーティングアルゴリズムによる3段階ディスパッチ

alarm_name から通知先チャンネルを3段階で解決し、全チャンネルに送信する。各段階でエントリが見つからない場合もフォールバックして処理を継続し、Lambda を正常終了させる。

段階と動作:

段階	処理	見つからない場合
Step 1	<code>_resolve_error_id(alarm_name, log_rows) → error_id</code>	WARNING ログ + "unknown" を返す
Step 2	<code>error-routing.yml</code> から <code>error_id</code> → <code>channel_ids</code>	WARNING ログ + ["discord"] にフォールバック
Step 3	<code>channel_ids</code> から Channel インスタンスを生成 (<code>email.*</code> は <code>group_id</code> を解析して生成)	WARNING ログ + そのチャンネルをスキップ
Dispatch	<code>channel.send(message)</code> を全チャンネルに実行	WARNING ログ + 次のチャンネルに継続

AC:

- AC-007-1: `_resolve_error_id("hdw-sakura", [])` は `"s3_data_missing"` を返す
- AC-007-1b: `_resolve_error_id("hdw-sakura", [log_row])` は `"lambda_failure"` を返す
- AC-007-2: `error_id` が `error-routing.yml` に存在しない場合、["discord"] にフォールバックし WARNING ログが出る
- AC-007-3: `channel_id` が解決できない場合、そのチャンネルをスキップし WARNING ログが出る
- AC-007-4: `Channel.send()` が例外を送出しても、次のチャンネルへの送信が継続される (AC-002-2 と同義)
- AC-007-5: いかなるフォールバック・例外が発生しても Lambda は正常終了する (エラーを再 raise しない)

スコープ外

- 重複通知の抑制ロジック (将来フェーズで対応)
- Discord 通知の廃止・変更 (本機能はメール通知の追加であり既存の Discord 通知は維持)
- メール開封率・クリック率のトラッキング
- メール配信リストの動的管理 UI (email.yaml の手動編集 + 再デプロイで管理する)
- alarm → error_id のマッピングの設定ファイル化 (コードに直書きで管理する)
- SES バウンス・苦情通知の自動処理

具体例・参考スキーマ

config/error-routing.yml

```
- id: s3_data_missing
  channels:
    - discord
```

```
- email.dev
- email.sakura
description: S3へのデータ不着を検知したとき (Lambda未起動)

- id: lambda_failure
channels:
  - discord
  - email.dev
  - email.sakura
description: Lambdaが起動したが処理中に失敗したとき
```

config/email.yaml

```
- id: dev
  add:
    - alerts@scrumsign.com

- id: sakura
  add:
    - captain@sakura-shipping.com
    - ops@sakura-shipping.com
```

Message dataclass (チャネル非依存)

```
@dataclass(frozen=True)
class Message:
    title: str          # Bedrock summary
    severity: str        # HIGH / MEDIUM / LOW
    confidence: str      # high / medium / low
    root_cause: str      # root_cause_hypothesis
    actions: list[str]   # suggested_actions
    alarm_name: str
    ship_name: str
    timestamp: datetime
```